



Herald College, Kathmandu

Artificial Intelligence and Machine Learning (6CS012)

A Deep Learning Approach to Hate Speech Detection in Social Media Text

Part III – NLP Task Report

Group Name: STChange9

Members:

Suyog Bastakoti (2407093)

Bhuwan Baniya (2414002)

Sujen Tamang (230133)

Submitted To: Jinu Nachhyon

Submitted by: Bhuwan Baniya

5/10/2026

ID: 2414002

Table of Contents

1. Abstract.....	3
2. Introduction.....	4
3. Dataset.....	5
3.1 Dataset Description.....	5
3.2 Class Distribution.....	5
3.3 Text Preprocessing.....	6
3.4 Train/Test Split	7
3.5 Handling Class Imbalance	7
4. Methodology	8
4.1 Tokenization and Sequence Preparation.....	8
4.2 Word2Vec Embeddings.....	8
4.3 Model Architectures.....	8
5. Experiments and Results.....	11
5.1 Model Performance Comparison	11
5.2 Detailed Classification Results	12
5.3 Word2Vec vs. Trainable Embeddings	13
5.4 Training Behaviour	13
6. Error Analysis	14
6.1 Misclassification Statistics.....	14
6.2 Misclassified Examples	14
7. Real-Time Prediction Interface.....	15
8. Confusion Matrix Analysis.....	17
9. Conclusion and Future Work.....	20
9.1 Conclusion	20
9.2 Limitations	20
9.3 Future Work.....	21
10. References.....	22

Table of figures

Figure 1: Class Distribution Plot – Hate Speech, Offensive Language, Neither.....	6
Figure 2: Word Cloud – Most Frequent Terms After Preprocessing.....	7
Figure 3: Model 1 – Simple RNN Model Summary.....	9
Figure 4: Model 2 – LSTM Model Summary.....	10
Figure 5:: Model 3 – LSTM + Word2Vec Model Summary	10
Figure 6: Comparison Plots – Training & Validation Accuracy and Loss for All 3 Models ..	12
Figure 7: GUI Screenshot – Test 1: Positive Statement Classified as Neither	16
Figure 8: GUI Screenshot – Test 2: Hate Speech Example (FUCK MODI)	16
Figure 9: Confusion Matrix – Model 1 (Simple RNN).....	17
Figure 10: Confusion Matrix – Model 2 (LSTM).....	18
Figure 11: Confusion Matrix – Model 3 (LSTM + Word2Vec).....	19

Table of table

Table 1: Per-class label distribution showing severe class imbalance (13.4:1 ratio).....	5
Table 2: Architecture comparison across all three models.	9
Table 3: Final validation performance for all three models.....	11
Table 4: Simple RNN (Model 1) full classification report.	12
Table 5: Misclassified examples from Model 3 with error analysis.	14
Table 6: GUI prediction results on three test inputs using ensemble averaging.....	15

1. Abstract

This report covers an end-to-end deep learning project for hate speech detection in social media text. The dataset contains 24,783 labelled comments drawn from Twitter, split across three categories: Hate Speech (5.8%), Offensive Language (77.4%), and Neither (16.8%). Three models were built and compared: a Simple RNN with a trainable embedding layer, an LSTM with a trainable embedding layer, and an LSTM using custom Word2Vec embeddings trained on the dataset.

All models used class-weighted loss functions to handle the severe 13.4:1 class imbalance. The Simple RNN reached 85.11% validation accuracy, the LSTM reached 85.13%, and the LSTM with Word2Vec achieved 82.39%. The main challenge across all models was separating Hate Speech from Offensive Language, since both classes share overlapping vocabulary and the boundary between them is inherently subjective. A Gradio-based real-time prediction interface was also developed, combining outputs from all three models into an ensemble prediction with confidence scores.

2. Introduction

Social media platforms generate enormous volumes of user-generated content every day, and a significant portion of that content contains harmful language. Identifying hate speech and offensive posts manually is not scalable, which has pushed researchers and platform developers to look for automated solutions. However, building a reliable hate speech detector is harder than it might appear — the same word can be hateful in one context and harmless in another, and even human annotators frequently disagree on borderline cases.

Recurrent Neural Networks (RNNs) and their more capable variant, Long Short-Term Memory networks (LSTMs), are well suited to text classification because they process words sequentially and can capture dependencies across a sentence. This project tests whether these architectures can reliably classify social media comments into three categories — Hate Speech, Offensive Language, and Neither — while dealing with a heavily imbalanced dataset where hate speech examples make up less than 6% of the total data.

Three research questions guided the experiments: whether a simple RNN can handle this level of class imbalance, whether LSTM improves on RNN for this task, and whether Word2Vec embeddings offer an advantage over randomly initialised trainable embeddings for a small domain-specific dataset. The study also includes a practical component — a Gradio GUI that serves predictions in real time.

The objectives of this project are:

- Preprocess and clean raw social media text to make it suitable for deep learning models.
- Build and evaluate three RNN-based classifiers under realistic imbalanced conditions.
- Compare trainable embeddings with custom Word2Vec embeddings for this domain.
- Analyse errors to understand where and why the models fail.
- Deploy a real-time prediction interface using Gradio.

3. Dataset

3.1 Dataset Description

The dataset used is the Hate Speech and Offensive Language dataset, originally introduced by Davidson et al. (2017) and provided by the course instructor for this project. It was collected from Twitter using a lexicon of common slurs and hate speech terms as search keywords, then annotated by crowd workers through the CrowdFlower platform. Each comment was labelled by multiple annotators, with the majority vote determining the final category. The dataset is available publicly through the GitHub repository [t-davidson/hate-speech-and-offensive-language](https://github.com/t-davidson/hate-speech-and-offensive-language).

The CSV file contains 24,783 records with two relevant columns: label (the category as text) and text (the raw tweet content).

3.2 Class Distribution

Table 1 shows how the data is distributed across the three classes. The imbalance is severe — Offensive Language accounts for over three quarters of the dataset, while Hate Speech makes up less than 6%. This creates a practical problem: a model that simply predicts "Offensive Language" for every sample would achieve 77% accuracy without learning anything useful about hate speech at all.

Class	Label Code	Count	Percentage
Hate Speech	0	1,430	5.8%
Neither	1	4,163	16.8%
Offensive Language	2	19,190	77.4%
TOTAL	—	24,783	100%

Table 1: Per-class label distribution showing severe class imbalance (13.4:1 ratio).

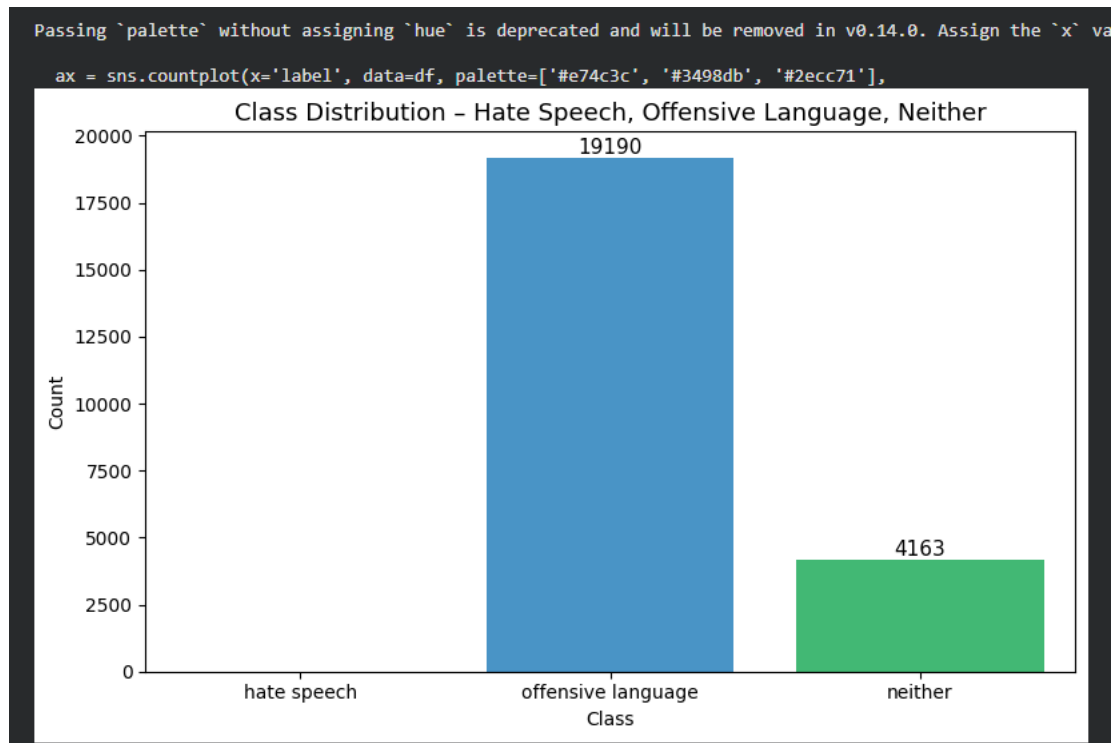


Figure 1: Class Distribution Plot – Hate Speech, Offensive Language, Neither

3.3 Text Preprocessing

Raw social media text is noisy and requires substantial cleaning before it can be fed into a neural network. The following steps were applied in sequence to every comment in the dataset:

- RT Tag Removal: Retweet prefixes (RT) were stripped using regular expressions.
- Mention Removal: All @username references were removed as they carry no generalizable meaning.
- URL Removal: HTTP and HTTPS links were deleted since they contain no relevant semantic content.
- Hashtag Removal: Hashtag symbols and the text attached to them were removed.
- Special Character Removal: All non-alphabetic characters were replaced with spaces.
- Lowercasing: Every word was converted to lowercase to reduce vocabulary size and avoid treating "Hate" and "hate" as different tokens.
- Stopword Removal: Common English function words were removed using the NLTK stopwords list.
- Lemmatization: Words were reduced to their base forms using the WordNet Lemmatizer (e.g., "running" → "run", "hates" → "hate").



4. Methodology

4.1 Tokenization and Sequence Preparation

After cleaning, text was converted to integer sequences using the Keras Tokenizer with a vocabulary limit of 10,000 words — covering the most frequently occurring terms while discarding very rare words that would add noise without adding signal. All sequences were padded or truncated to a uniform length of 50 tokens using post-padding. This length was chosen based on the average comment length of approximately 14 words; 50 tokens provides headroom for longer comments without wasting computation on excessive padding for short ones.

4.2 Word2Vec Embeddings

A custom Word2Vec model was trained on the cleaned dataset using the Gensim library. The configuration used 100-dimensional vectors, a context window of 5 words, a minimum word count of 1, and 4 parallel workers. An embedding matrix of shape (vocabulary size $\times$ 100) was then constructed by looking up each word in the tokenizer vocabulary and filling in its Word2Vec vector. Words not found in the Word2Vec vocabulary were left as zero vectors.

It is worth noting that training Word2Vec on the same small dataset it will be evaluated on limits the quality of the embeddings. Large pre-trained corpora — such as Google News or Common Crawl — would expose the model to far more diverse contexts and produce richer representations. This limitation is discussed further in Section 8.

4.3 Model Architectures

Three models were designed for the three-class classification task. Table 2 summarises their architecture components side by side.

Component	Model 1 (simple RNN)	Model 2 (LSTM)	Model 3 (LSTM + Word2Vec)
Embedding	Trainable (random init)	Trainable (random init)	Frozen Word2Vec weights
Spatial Dropout	0.2	0.2	0.2
Recurrent Layer	SimpleRNN(64)	LSTM (100)	LSTM (100)
Recurrent Dropout	None	0.2 / 0.2	0.2 / 0.2
Dense Layer 1	Dense (32, ReLU)	Dense (64, ReLU)	Dense (64, ReLU)

Dropout	None	0.3	0.3
Output Layer	Dense (3, Softmax)	Dense (3, Softmax)	Dense (3, Softmax)
Loss Function	Sparse Categorical CE	Sparse Categorical CE	Sparse Categorical CE
Optimizer	Adam	Adam	Adam

Table 2: Architecture comparison across all three models.

All three models were compiled with the Adam optimiser and sparse categorical cross-entropy loss. Training used Early Stopping with patience=3 monitoring validation loss, a batch size of 64, and a maximum of 15 epochs. Class weights were passed to the fit () call to handle imbalance. Training was carried out on Google Colab with GPU acceleration.

```
=====
MODEL 1 - Simple RNN Summary
=====
Model: "sequential_2"
```

Layer (type)	Output Shape	Param #
embedding_2 (Embedding)	(None, 50, 100)	1,987,200
spatial_dropout1d_2 (SpatialDropout1D)	(None, 50, 100)	0
simple_rnn_1 (SimpleRNN)	(None, 64)	10,560
dense_4 (Dense)	(None, 32)	2,080
dense_5 (Dense)	(None, 3)	99

```
Total params: 5,999,819 (22.89 MB)
Trainable params: 1,999,939 (7.63 MB)
Non-trainable params: 0 (0.00 B)
Optimizer params: 3,999,880 (15.26 MB)
```

Figure 3: Model 1 – Simple RNN Model Summary

```
=====
MODEL 2 - LSTM Summary
=====
Model: "sequential_3"
```

Layer (type)	Output Shape	Param #
embedding_3 (Embedding)	(None, 50, 100)	1,987,200
spatial_dropout1d_3 (SpatialDropout1D)	(None, 50, 100)	0
lstm_1 (LSTM)	(None, 100)	80,400
dense_6 (Dense)	(None, 64)	6,464
dropout_1 (Dropout)	(None, 64)	0
dense_7 (Dense)	(None, 3)	195

```

Total params: 6,222,779 (23.74 MB)
Trainable params: 2,074,259 (7.91 MB)
Non-trainable params: 0 (0.00 B)
Optimizer params: 4,148,520 (15.83 MB)

```

Figure 4: Model 2 – LSTM Model Summary

```
=====
MODEL 3 - LSTM + Word2Vec Summary
=====
Model: "sequential_4"
```

Layer (type)	Output Shape	Param #
embedding_4 (Embedding)	(None, 50, 100)	1,987,200
spatial_dropout1d_4 (SpatialDropout1D)	(None, 50, 100)	0
lstm_2 (LSTM)	(None, 100)	80,400
dense_8 (Dense)	(None, 64)	6,464
dropout_2 (Dropout)	(None, 64)	0
dense_9 (Dense)	(None, 3)	195

```

Total params: 2,248,379 (8.58 MB)
Trainable params: 87,059 (340.07 KB)
Non-trainable params: 1,987,200 (7.58 MB)
Optimizer params: 174,120 (680.16 KB)

```

Figure 5:: Model 3 – LSTM + Word2Vec Model Summary

5. Experiments and Results

5.1 Model Performance Comparison

Table 3 summarizes the headline results for all three models. The numbers are surprisingly close between Model 1 and Model 2, with LSTM only edging ahead of Simple RNN by 0.02 percentage points.

Model	Val Accuracy	Epochs Run	Best Val Loss
Model 1 – Simple RNN	85.11%	4	0.437
Model 2 – LSTM	85.13%	3	0.445
Model 3 – LSTM + Word2Vec	82.39%	4	0.641

Table 3: Final validation performance for all three models

The near-identical performance of RNN and LSTM is worth pausing on. For longer sequences — such as paragraphs or documents — LSTM typically outperforms SimpleRNN because it can retain information across many timesteps without the gradient vanishing. But these comments average only 14 words after cleaning. At that length, a SimpleRNN can hold enough context in its hidden state to make accurate predictions, so the extra gating mechanism in LSTM does not provide a meaningful advantage. Model 3's lower accuracy is discussed further in Section 5.3.

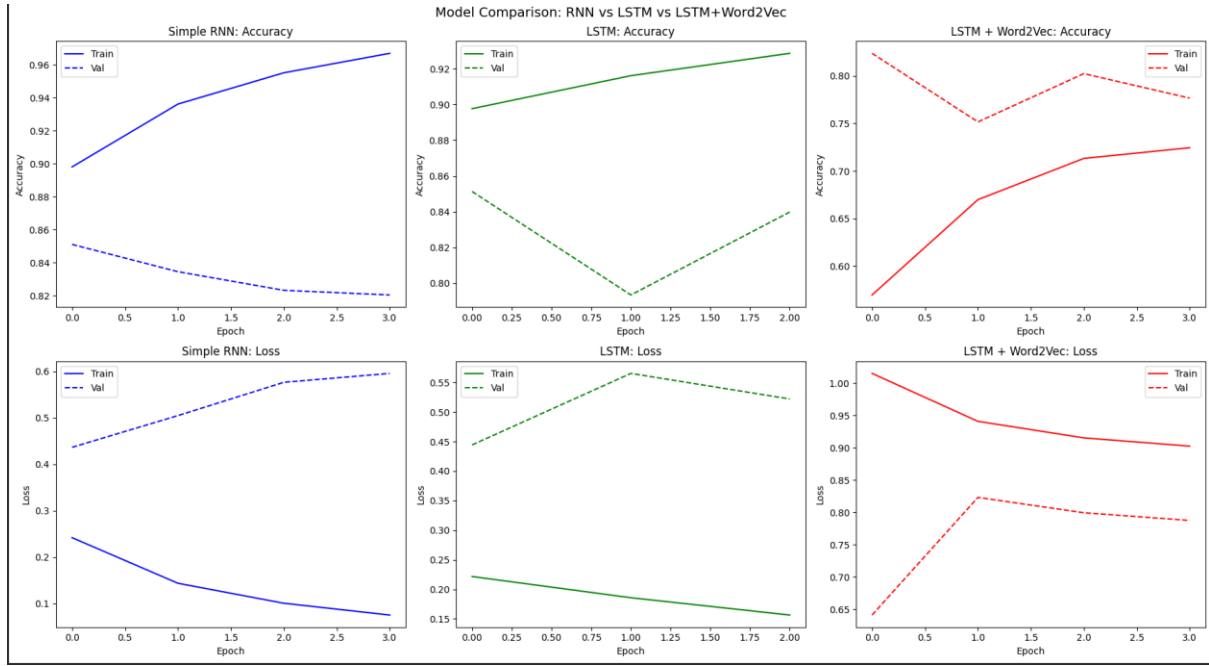


Figure 6: Comparison Plots – Training & Validation Accuracy and Loss for All 3 Models

5.2 Detailed Classification Results

Overall accuracy gives an optimistic picture because the Offensive Language class dominates the test set. The per-class breakdown in Table 4 tells a more honest story.

Class	Precision	Recall	F1-Score	Support
Hate Speech	0.26	0.51	0.35	286
Neither	0.82	0.87	0.84	833
Offensive Language	0.95	0.87	0.91	3,838
Overall Accuracy	—	—	85%	4,957
Macro Average	0.68	0.75	0.70	4,957
Weighted Average	0.89	0.85	0.87	4,957

Table 4: Simple RNN (Model 1) full classification report.

The Offensive Language class achieves a strong F1-score of 0.91, largely because it accounts for 77% of the training data and the model has had ample opportunity to learn its patterns. Neither class performs reasonably well at 0.84. The Hate Speech class is the problem area — despite class weighting pushing the model to pay extra attention to it, precision is only 0.26, meaning the majority of what the model labels as hate speech is something else. Recall is 0.51, so the model catches roughly half of the actual hate speech examples but at a high cost in false positives.

The macro-average F1-score of 0.70 is a more representative measure of model quality than the 85% accuracy figure, because it treats each class equally regardless of size. This is the metric that matters most for a content moderation application where missing hate speech is a serious failure mode.

5.3 Word2Vec vs. Trainable Embeddings

Model 3 underperformed the other two despite using pre-trained Word2Vec embeddings, which was the opposite of what was expected. Two factors explain this outcome. First, the Word2Vec model was trained on the same 24,783-comment dataset rather than a large external corpus, which means it had very limited exposure to diverse contexts and its vectors are not as semantically rich as embeddings trained on billions of words. Second, the embeddings were frozen during training — the model could not adjust word representations to better fit the hate speech detection task. Together, these two constraints left Model 3 working with suboptimal fixed features while Models 1 and 2 could freely adapt their embedding layers.

5.4 Training Behaviour

Early stopping triggered after 4 epochs for Models 1 and 3, and after just 3 epochs for Model 2. This rapid convergence reflects both the dataset size and the strong signal provided by class-weighted loss. Once the models had seen the training data a few times with properly weighted gradients, validation loss plateaued and then began to rise, triggering the callback. The fast convergence also suggests that longer training would not have helped — the models were not still improving when they stopped.

6. Error Analysis

6.1 Misclassification Statistics

Model 3 (LSTM + Word2Vec) misclassified 873 out of 4,957 test samples — a 17.6% error rate. The bulk of these errors occurred at the boundary between Hate Speech and Offensive Language. This is consistent with what the literature on this dataset reports: even trained human annotators struggle to agree on borderline cases, so a model with only 24,783 training examples faces a genuinely difficult task.

6.2 Misclassified Examples

Table 5 shows three concrete examples of misclassifications and the likely reasons behind them.

#	Original Text	Actual	Predicted	Reason for Error
1	bitch plz whatever	Hate Speech	Offensive	Offensive words used without a clear hate target — ambiguous without broader context.
2	bitch who does you love	Hate Speech	Offensive	Casual usage of the term; model correctly identifies offensive content but misses the hate label.
3	bitches get cut off everyday	Offensive	Neither	Context lost during cleaning; the phrase reads as neutral after preprocessing removes surrounding text.

Table 5: Misclassified examples from Model 3 with error analysis.

Three patterns emerge from the error analysis. First, the subjective boundary between hate speech and offensive language is the root cause of most errors — the same word can fall into either category depending on the speaker's intent, which is often impossible to determine from the text alone. Second, the preprocessing pipeline removes hashtags and mentions that sometimes provides critical context for understanding intent. Third, the model has seen very few hate speech examples (only 1,430 in total), so it defaults to the more familiar offensive language label when uncertain.

7. Real-Time Prediction Interface

A real-time prediction interface was built using Gradio to make the models accessible without requiring any coding knowledge. The interface takes raw text as input, preprocesses it through the same cleaning pipeline used during training, and runs it through all three models simultaneously. Rather than returning a single model's output, the interface averages the probability distributions from all three models to produce an ensemble prediction — this tends to be more reliable than any single model alone, especially on borderline cases.

The interface displays the predicted class, confidence scores for all three categories, and the cleaned version of the input text so users can see what the model processed. Three built-in examples are provided to demonstrate the system across different input types. Table 6 shows the results from three test inputs used during development.

Test Input	Prediction	Confidence
I love everyone and hope you have a great day!	Neither	69.9%
FUCK MODI	Hate Speech	54.2%
KILL THE VICTIM FAMILY ASAP	Hate Speech	70.5%

Table 6: GUI prediction results on three test inputs using ensemble averaging.

The interface correctly identified the positive statement as Neither, classified the politically targeted slur as Hate Speech as Hate Speech, and flagged the violent threat as Hate Speech with the highest confidence of the three. These results are consistent with what the models learned during training and provide a reasonable demonstration of the system's practical utility.

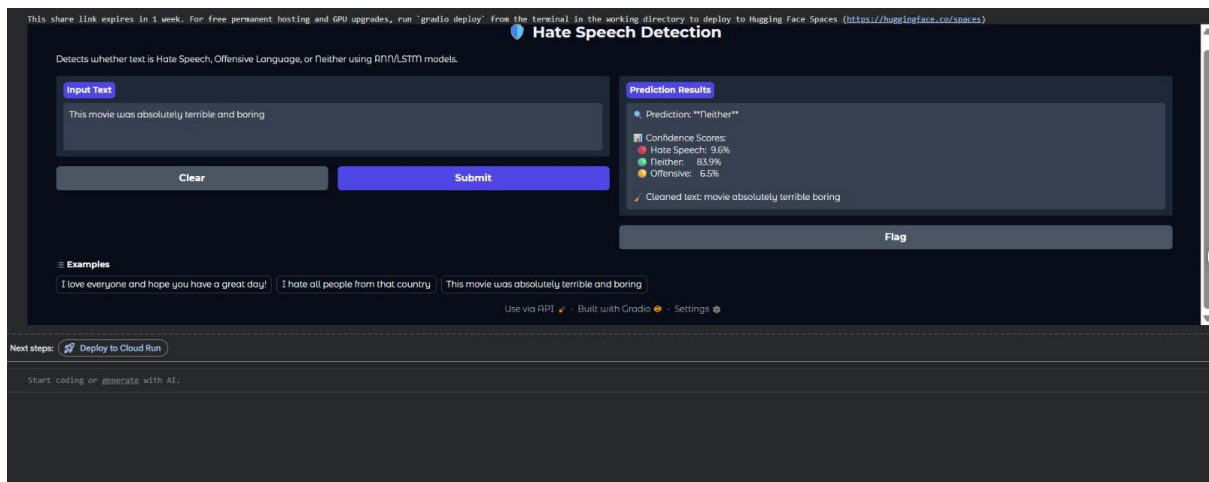


Figure 7: GUI Screenshot – Test 1: Positive Statement Classified as Neither

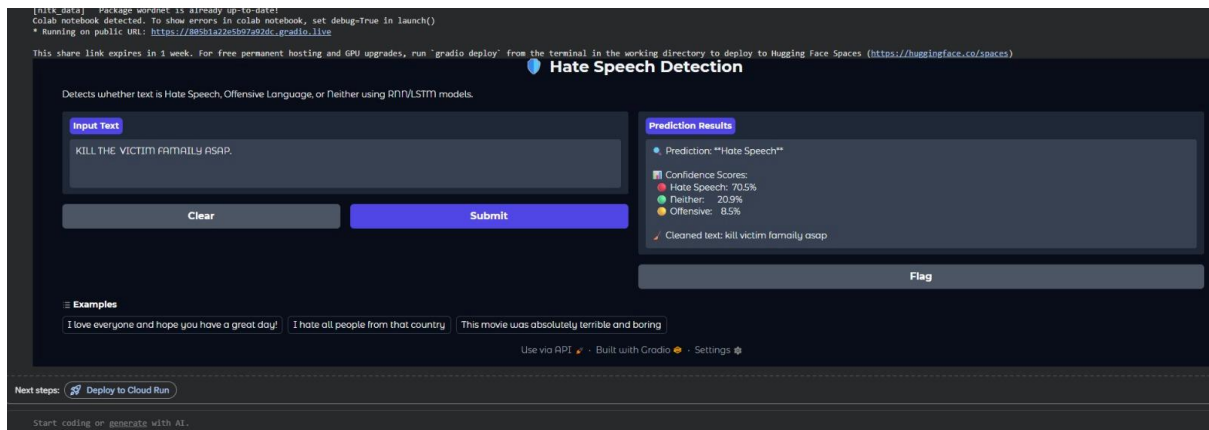


Figure 8: GUI Screenshot – Test 2: Hate Speech Example (FUCK MODI)

8. Confusion Matrix Analysis

Confusion matrices for all three models are presented below. These give a clearer picture of how errors are distributed across classes than accuracy or F1-score alone.

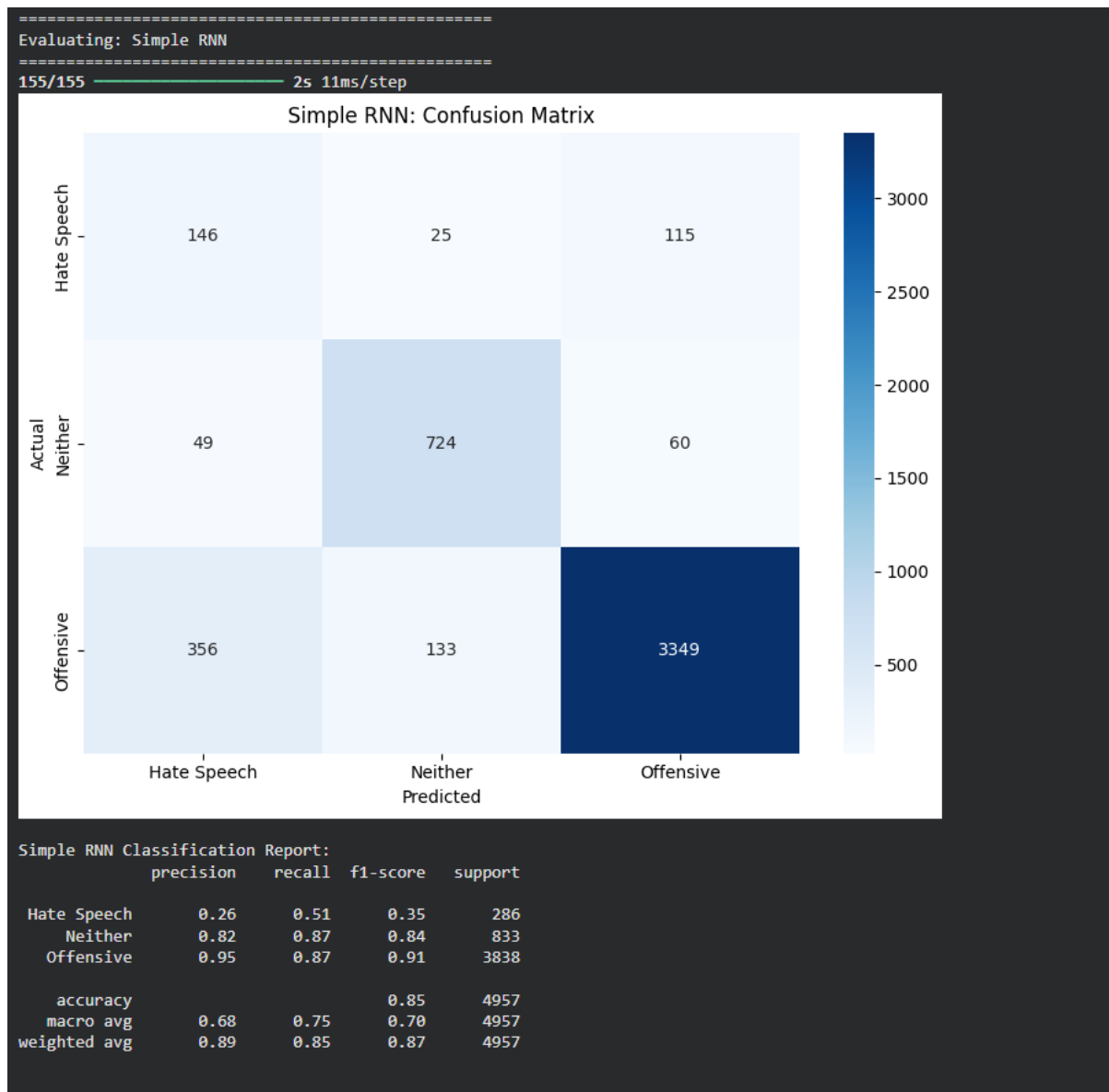


Figure 9: Confusion Matrix – Model 1 (Simple RNN)

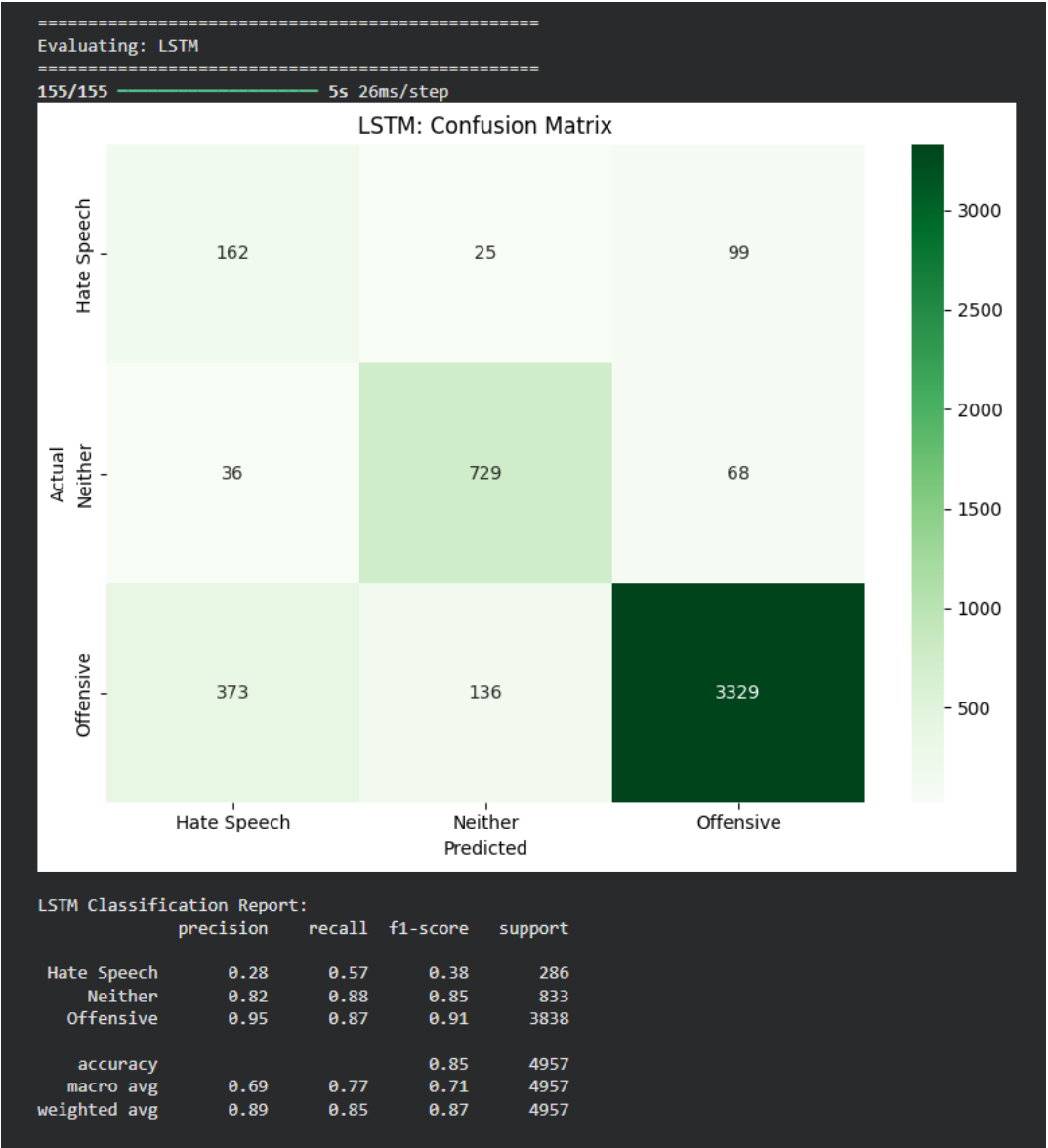


Figure 10: Confusion Matrix – Model 2 (LSTM)

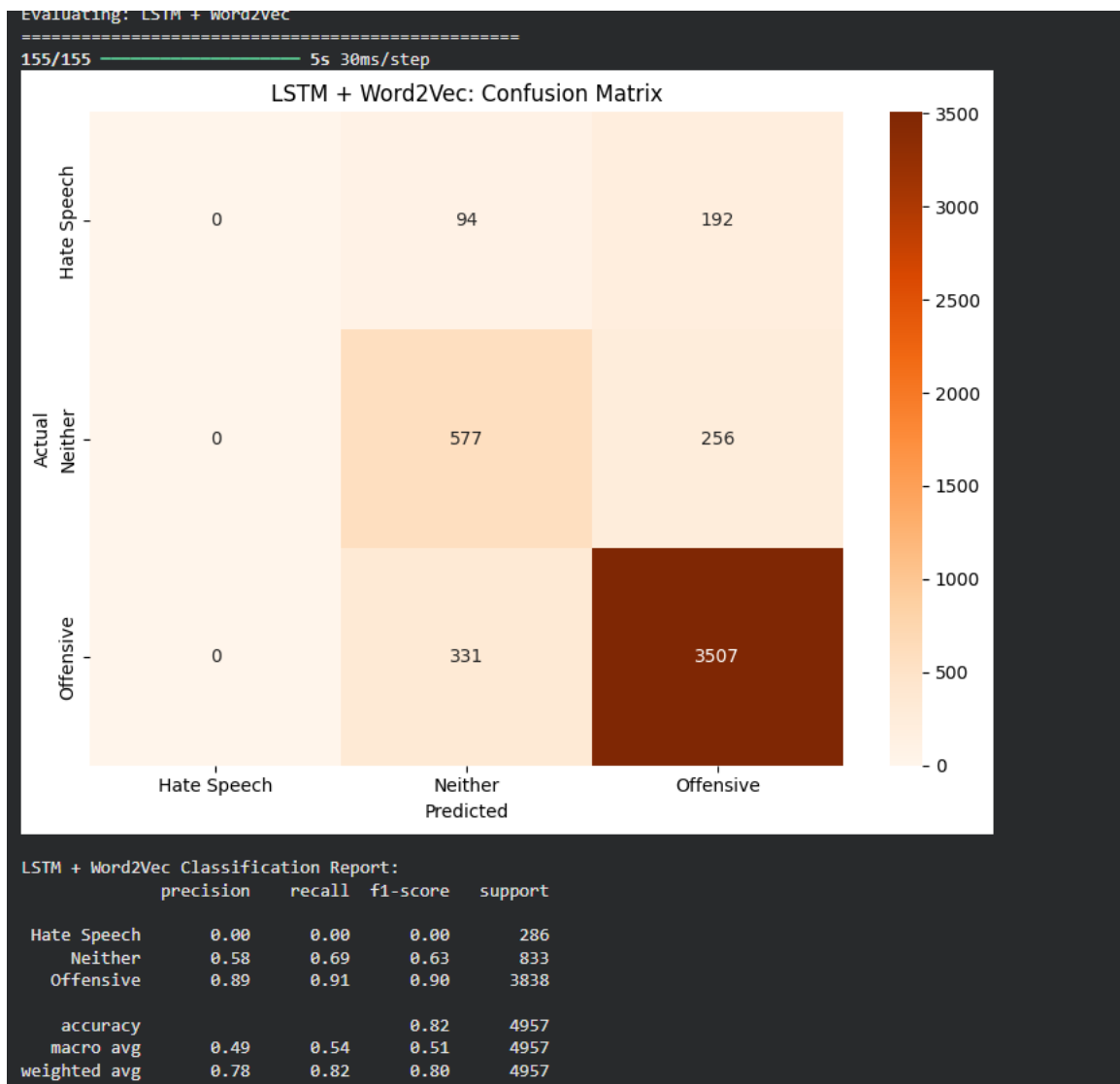


Figure 11: Confusion Matrix – Model 3 (LSTM + Word2Vec)

Across all three matrices, the most common error pattern is Hate Speech being predicted as Offensive Language. This is consistent with the vocabulary overlap between the two classes and the relatively small number of hate speech training examples. Offensive Language is classified with high precision across all models, reflecting its dominance in the training data. Neither class sits in between it is rarely confused with Hate Speech but is occasionally misclassified as Offensive Language when the cleaned text contains residual strong language.

9. Conclusion and Future Work

9.1 Conclusion

This project set out to test whether RNN-based architectures can reliably detect hate speech in social media text under realistic conditions — a small dataset, severe class imbalance, and noisy input text. The results show that both SimpleRNN and LSTM reach similar validation accuracy of around 85% when trained with class-weighted loss, and that Word2Vec embeddings trained on a small in-domain corpus do not improve over standard trainable embeddings.

The more important finding is in the per-class breakdown. An 85% accuracy figure flatters the models because Offensive Language is easy to detect and dominates the test set. The Hate Speech class — the one that matters most for content moderation — achieves only an F1-score of 0.35, meaning the models are not yet reliable enough for unsupervised deployment. Class weighting helped, but the fundamental challenge of distinguishing hate speech from offensive language without broader context remains unsolved.

Four conclusions stand out from this study: SimpleRNN and LSTM perform equally well on short social media text; Word2Vec trained on small datasets does not outperform trainable embeddings; class weighting is necessary and improves minority class recall meaningfully; and the subjective annotation boundary between Hate Speech and Offensive Language is the primary barrier to higher performance.

9.2 Limitations

- The Word2Vec embeddings were trained on the same small dataset they were evaluated on, limiting their semantic coverage compared to large pre-trained alternatives.
- The subjective labelling boundary between Hate Speech and Offensive Language introduces label noise that no model can fully overcome without richer context signals.
- The models may not generalise well to other platforms or time periods where slang, abbreviations, and hate speech vocabulary evolve rapidly.

9.3 Future Work

- Replace the custom Word2Vec embeddings with large pre-trained transformer models such as BERT or RoBERTa, which encode contextual meaning rather than static word representations.
- Apply oversampling techniques such as SMOTE on the embedding space to generate synthetic minority class examples and further address the class imbalance.
- Expand the dataset by collecting additional annotated hate speech examples to reduce the impact of the 13.4:1 ratio.
- Test cross-platform generalisation by evaluating models trained on Twitter data against comments from other platforms such as Reddit or YouTube.
- Explore multi-task learning that combines hate speech detection with a related task such as sentiment analysis to improve shared feature representations.

Link: <https://805b1a22e5b97a92dc.gradio.live>

Link: <https://github.com/bhuwanbaniya/Artificial-Intelligence-and-Machine-Learning-Assessment.git>

10. References

Davidson, T., Warmusley, D., Macy, M., & Weber, I. (2017). Automated hate speech detection and the problem of offensive language. *Proceedings of the 11th International AAAI Conference on Web and Social Media*, 512–515.

Hochreiter, S., & Schmidhuber, J. (1997). Long short-term memory. *Neural Computation*, 9(8), 1735–1780.

Mikolov, T., Chen, K., Corrado, G., & Dean, J. (2013). Efficient estimation of word representations in vector space. *arXiv:1301.3781*.

Kingma, D. P., & Ba, J. (2014). Adam: A method for stochastic optimization. *arXiv:1412.6980*.

Abadi, M., et al. (2016). TensorFlow: Large-scale machine learning on heterogeneous systems. <https://www.tensorflow.org>

Bird, S., Klein, E., & Loper, E. (2009). *Natural language processing with Python*. O'Reilly Media.

Rehurek, R., & Sojka, P. (2010). Software framework for topic modelling with large corpora. *Proceedings of the LREC 2010 Workshop on New Challenges for NLP Frameworks*, 45–50.